

Drift — Take-Home Assignment

Build the Robot. Build the Home. Clean It Up.

Overview

This assignment evaluates your ability to independently set up, build, and run a complete robotics simulation from scratch. You will build a specific robot model, design a home environment, and execute a tidying task — all running locally on your machine. We are **not** evaluating perception or control algorithm sophistication. We are evaluating your fluency with simulation tooling: URDF/SDF authoring, world construction, launch orchestration, and your ability to produce a simulation that actually runs and does something meaningful.

Required Environment

Your simulation must build and run on a standard Linux system (Ubuntu 22.04 or 24.04) with the following:

Component	Requirement
OS	Ubuntu 22.04 / 24.04
ROS Distribution	ROS 2 Humble or Jazzy
Simulator	Gazebo (Classic 11 or Harmonic), MuJoCo, Isaac Sim, or any open-source physics simulator
Build system	colcon (standard ROS 2 workspace)

The Assignment

Build a **complete, runnable simulation** of a home-tidying robot. The robot navigates a two-room home, detects small objects scattered around, picks them up, and returns them to a collection box. All three components below must work together as an integrated system.

Component 1 — The Robot

Build the following robot as a URDF or SDF model. The design is specific — you are implementing this spec, not choosing your own robot:

Base & Mobility

- A **4-wheeled differential or skid-steer base**. All 4 wheels must be actuated and make contact with the ground.
- The base must be appropriately sized to navigate through doorways and around furniture (roughly 0.4–0.6m wide).

Torso & Arm

- A vertical **torso** mounted on the base.
- A **3-DOF actuated arm** (3 revolute joints) mounted on one side of the torso — shoulder, elbow, and wrist. The arm must be capable of reaching the ground plane and lifting small objects.
- A simple **gripper or end-effector** at the wrist (can be a fixed scoop, parallel jaw, or magnetic — your choice, but it must be modeled).
- A second **arm on the opposite side** of the torso — this can be a static/cosmetic arm or a functional one. Both arms must be visually present in the model.

Head / Face

- A **flat rectangular panel** on top of the torso representing an LED display face. This is a visual-only element — model it as a flat box or plane with a distinct color or texture.

Sensors

- A **camera** mounted on the torso (not the head) facing forward, publishing image data to a ROS 2 topic. This is the robot's primary navigation sensor.
- At least **one additional sensor** of your choice (LiDAR, IMU, bumper/contact sensor, depth camera, ultrasonic, etc.) that publishes to a ROS 2 topic.

Physical Properties

- All links must have **physically plausible** mass, inertia, collision, and visual properties. Placeholder values (mass=1.0, identity inertia matrix) will be flagged.
- Joint limits, damping, and friction must be set so the robot does not explode, collapse, or drift on spawn.

You may reference open-source URDFs for structural guidance, but this robot design is custom — assemble it yourself. Document any external resources you used.

Component 2 — The Home

Design a simulation world (SDF or Gazebo world file) representing a small home. The environment must include:

- **Two rooms** connected by a doorway or open passage wide enough for the robot to pass through.
- **Walls** enclosing both rooms with appropriate collision boundaries.
- **Furniture** in each room — at minimum: a table, chairs, and one additional piece (shelf, couch, cabinet, etc.). Objects should constrain the robot's navigation path.
- **Small objects scattered around** — at least **5 small pickup-able objects** placed on the floor across both rooms (e.g., blocks, cylinders, small boxes representing toys, cans, shoes, etc.). These are the items the robot must collect.
- A **large open box** placed near the robot's starting position in one of the rooms. This is the collection target — objects go here.
- A ground plane with **appropriate friction** (not zero-friction default).
- Reasonable **lighting and physics configuration** (gravity, step size).

Stock Gazebo models (tables, chairs, etc.) are fine. Bonus if you create custom meshes or use a coherent visual theme, but this is not required.

Component 3 — The Tidying Task

Implement the robot's behavior for the tidying task. This has two tiers:

Minimum requirement (must have):

- The robot **autonomously navigates** through both rooms of the home without colliding with walls or furniture. Simple logic is fine — hard-coded waypoints, wall-following, random exploration with obstacle avoidance — we are not grading the navigation algorithm.
- Navigation must be **launched with a single command** (no manual joystick or teleop during the test).
- The robot must **publish sensor data** (camera images, LiDAR scans, etc.) throughout the run.
- The run must produce **measurable output** — logged odometry, a map of the path taken, or a terminal printout of rooms visited / distance covered.
- The simulation must run to completion in **under 5 minutes** of simulation time.

Bonus (strong plus, not required):

- The robot **detects** one or more small objects on the ground (via camera, LiDAR proximity, or known positions).
- The robot **picks up** an object using its arm and gripper (even a simple "attach on contact" approach counts).
- The robot **returns to the box** and **places the object** inside it.
- Any level of pick-and-place — even one object, even with hardcoded positions — is a significant differentiator.

We are grading whether the full pipeline works end-to-end: spawn the robot in the home, run the task, observe results. A robot that competently drives through both rooms and logs its journey is a solid pass. A robot that also picks up even one object and brings it back is exceptional.

Component 4 — Beat Drift

Drift (godrift.ai) is our AI copilot for robotics simulation engineers. It currently works with ROS 2 and Gazebo. Before you submit, you will use the Drift CLI to generate its own version of the robot model and the home world from the same spec above. Then you will include a head-to-head comparison in your approach document.

Your build should be better than what Drift produces. If an AI copilot can match or beat your simulation work, we don't have a reason to hire for this role. We are looking for engineers whose craft, judgment, and debugging instincts produce results that our tool cannot — yet. The whole point of joining Drift is to close that gap.

What to do:

- Install the Drift CLI by running **`curl -fsSL https://godrift.ai/install | bash`** and set it up in your ROS 2 + Gazebo workspace.
- Use Drift to generate a robot URDF and a home world SDF based on the same spec described in Components 1 and 2 above. Give it the same constraints you worked with.
- Attempt to launch Drift's generated output in Gazebo. Note what works and what doesn't.
- In your **approach document**, include a dedicated section titled **"My Build vs. Drift's Build"** comparing the two side by side.

We will run the same comparison internally. If your robot model and world are not meaningfully better than what Drift generates today, that is a clear signal. The engineers we are hiring are the ones who will make Drift better — and that starts with being better than it.

Evaluation Criteria

This rubric is weighted to reflect what matters for a simulation engineer at Drift. Notice that controls, perception, and algorithm quality are explicitly not the focus.

Criterion	Weight	What We Look For
It runs	30%	We clone your repo, follow your README, and the simulation launches without errors on a clean Ubuntu + ROS 2 system. This is pass/fail and it is the single most important criterion.
Robot model quality	20%	Matches the spec: 4 wheels, torso, two arms (one 3-DOF), face panel, torso camera. Physically plausible inertials. Sensors publish. No visual-only links pretending to be functional.
Home world design	15%	Two rooms with doorway. Furniture placed intentionally. Small objects scattered realistically. Collection box present. Physics properties are reasonable.
Autonomous navigation	10%	Robot moves through both rooms without collision. Launched with a single command. Produces measurable output. Runs under 5 min sim time.

Pick & place (bonus)	(+10%)	Any level of object pickup and return to box. Even one object with hardcoded positions counts. This is additive — it can push you above 100%.
Beat Drift	15%	Your robot and world are demonstrably better than Drift CLI output. Comparison is concrete and specific. You clearly understand where AI-generated sim artifacts fall short.
Documentation & approach	5%	Approach document explains design choices, tradeoffs, debugging stories, what you'd improve. README gets us from zero to running sim.
Code quality & structure	5%	Clean ROS 2 package structure. Sensible file naming. No hardcoded absolute paths. Launch files are parameterized where appropriate.

Deliverables

1. GitHub Repository

- A public or private GitHub repo (if private, grant access to <https://github.com/drift-tech>).
- Must contain a valid ROS 2 workspace with your package(s).
- A **README.md** with exact setup and launch instructions. Assume the reviewer has a clean Ubuntu 22.04/24.04 + ROS 2 Humble/Jazzy install and nothing else. List every dependency.
- Include a **requirements.txt** or equivalent for any Python/system dependencies beyond ROS 2 defaults.

2. Demo Video

- **2–4 minutes.** Screen recording of the simulation running end-to-end.
- Must show: the launch command, the robot spawning in the home, the robot navigating both rooms, and any pick-and-place attempts.
- Include a brief text overlay explaining what is happening. No editing or polish needed.
- Upload to YouTube (unlisted), Google Drive, or Loom. Include the link in your README.

3. Approach Document

- A **1–3 page** PDF or Markdown file included in the repo (docs/APPROACH.md or docs/approach.pdf).
 - Cover: how you built the robot model, how you designed the home layout, your navigation strategy, what tradeoffs you made, and what you would improve with more time.
 - Mention any debugging challenges you hit and how you resolved them — this is where real simulation experience shows.
 - If you used any existing models or assets as a base, clearly state the source.
-

We are looking for engineers who can make simulations work — who know the difference between a URDF that renders and one that simulates, who can debug a robot that spawns and immediately falls through the floor, and who can build a world that feels like a place. Show us that.

Good luck. We are excited to see what you build.

— *The Drift Team* | godrift.ai